CSE 210: Computer Architecture
Lecture 22: Floating Point

Stephen Checkoway

Slides from Cynthia Taylor

CS History: IBM 704 Data-Processing Machine



Man and woman working with IBM type 704 electronic data processing machine used for making computations for aeronautical research. By NASA, Public Domain

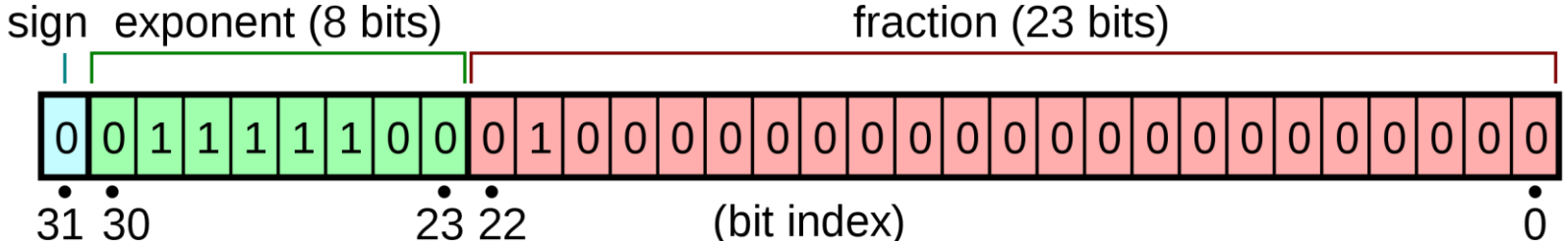
- First mass-produced computer with floating point arithmetic
- Introduced in 1954
- Had 36 bit words
- Floating point had
  - 1 sign bit
  - 8 bit exponent (biased by 127)
  - **27** bit fraction (no hidden bit)
- "pretty much the only computer that could handle complex math" at the time
- (Lisp was originally implemented on the IBM 704. Car and cdr come from this machine.)

Review

- Unsigned 32-bit integers let us represent 0 to $2^{32} - 1$
- Signed 32-bit integers let us represent -2^{31} to $2^{31} - 1$
- 32-bit floating point numbers let us represent a wider range of values: larger, smaller, fractional

$$(-1)^s * 1.x * 2^e$$

- 1 bit for sign s (1 = negative, 0 = positive)
- 8 bits for exponent e + 127
- 0 bits for implicit leading 1 (called the “hidden bit”)
- 23 bits for significand (without hidden bit)/fraction/~~mantissa~~ x



Special Cases

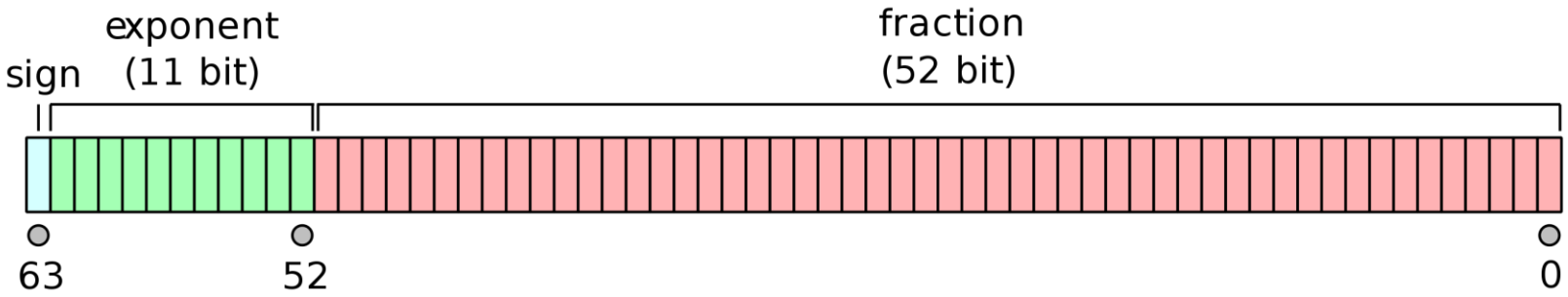
	Exponent	Fraction
Zero	0	0
Subnormal	0	Nonzero
Infinity	255	0
NaN	255	Nonzero

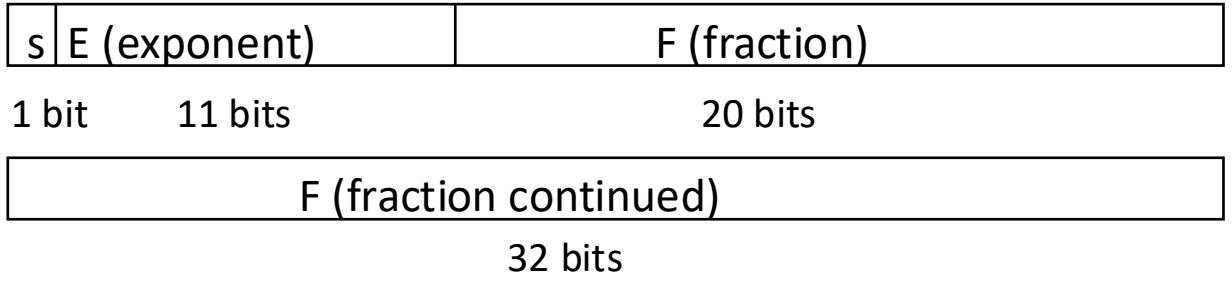
- Subnormal number: Numbers with magnitude smaller than 2^{-126}
  - They have an implicit leading 0 bit and an exponent of -126
- NaN: Not a Number. Results from 0/0, 0 * ∞, (+∞) + (–∞) , etc.

Overflow/underflow

- Overflow happens when a positive exponent becomes too large to fit in the exponent field
- Underflow happens when a negative exponent becomes too large (in magnitude) to fit in the exponent field
- One way to reduce the chance of underflow or overflow is to offer another format that has a larger exponent field
  - Double precision – takes two 32-bit words

Double precision in IEEE Floating Point





Floats in higher-level languages

- C, Java: float, double
- JavaScript: numbers are always 64-bit double precision
- Rust: f16, f32, f64
  - f16 is a 16-bit “half precision” floating point type, support is currently experimental
- Sometimes intermediate values (e.g., x*y in x*y + z) may be doubles (or larger types!) even when the inputs are all floats

Which of these numbers does not exist in JavaScript?

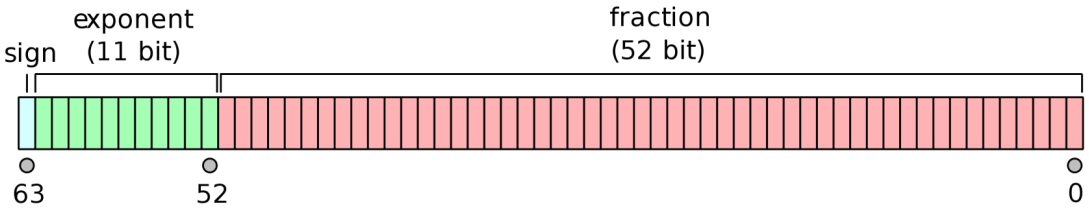
A. 9007199254740991

B. 9007199254740992

C. 9007199254740993

D. 9007199254740994

E. More than one of the above



Hint: 9007199254740992 is 2^{53}

There are always 2^{52} evenly spaced doubles between 2^n and 2^{n+1}. How many **floats** will there be between 2^n and 2^{n+1}?

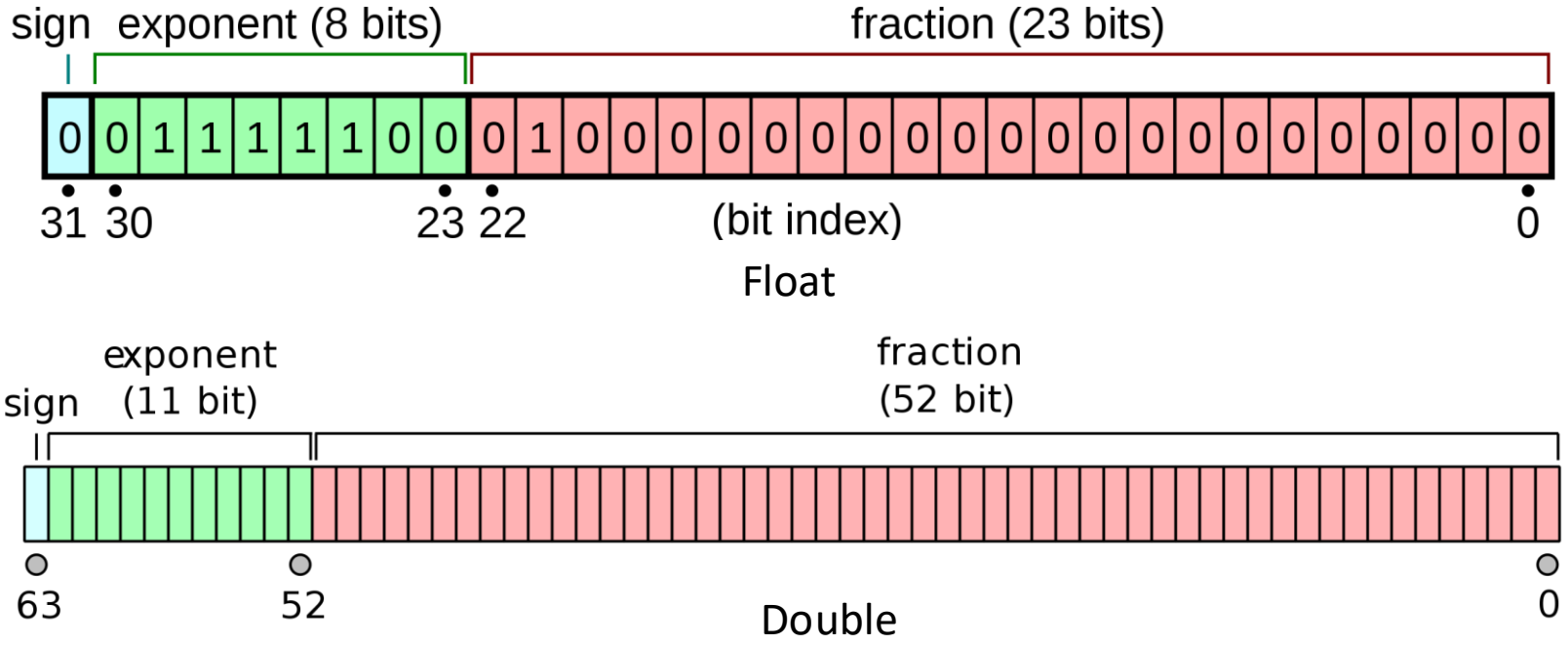
A. 2^8

B. 2^{23}

C. 2^{32}

D. 2^{52}

E. None of the above



Weird Float Tricks

- For floats of the same sign:
  - Adjacent floats have adjacent integer representations
  - Incrementing the integer representation of a float moves to the next representable float, moving away from zero
- This is specific to the IEEE 754 implementation of floating point!
- Want to play around with floats?
  - https://float.exposed/

Adding floating point numbers together

1. Denormalize inputs so both have the larger exponent by shifting the input with the smaller exponent to the right (shift the bits of the significand to the right) until they have the same exponent
2. Add the significands together, taking the sign into account
3. Normalize the result by shifting the significand left or right as necessary to have a single 1 bit to the left of the binary point

Adding in floating point (assuming 4 fractional bits)

Add together $1.1011 * 2^0$ and $1.0110 * 2^2$

1. Denormalize so both have the larger exponent
   $0.0110 * 2^2 + 1.0110 * 2^2$
2. Add significands taking sign of numbers into account
   $1.1100 * 2^2$
3. Normalize to a single leading 1 (nothing to do for this example because it already has a single leading 1)
   $1.1100 * 2^2$

We got $1.1011 * 2^0 + 1.0110 * 2^2 = 1.1100 * 2^2$

$1.1011 * 2^0 = 1.6875$

$1.0110 * 2^2 = 5.5$

$1.1100 * 2^2 = 7.0$

But $1.6875 + 5.5 = 7.1875$

Is this the correct result?

A. Yes [explain the discrepancy between 7.0 and 7.1875]

B. No [why not? Is there a more correct result?]

C. I have no idea what is going on, please explain more!

We got the wrong result!

$1.1011 * 2^0 + 1.0110 * 2^2 = 1.1100 * 2^2$ isn't the correct result because there's a floating point value with 4 fractional bits that's closer to the correct answer

$1.1101 * 2^2 = 7.25$ which is closer to the correct result, 7.1875, than 7.0 is

Why did we get the wrong result?

Add together $1.1011 * 2^0$ and $1.0110 * 2^2$

1. Denormalize so both have the larger exponent: $0.0110 * 2^2 + 1.0110 * 2^2$
2. Add significands taking sign of numbers into account: $1.1100 * 2^2$
3. Normalize to a single leading digit: $1.1100 * 2^2$

A. The algorithm was wrong!
B. We lost some bits in step 1
C. We lost some bits in step 2
D. We lost some bits in step 3
E. Uh…

The fix is to use more bits for the shifted significands in step 1. Let's use 8 bits.

Add together $1.1011 * 2^0$ and $1.0110 * 2^2$

1. Denormalize so both have the larger exponent: $0.01101110 * 2^2 + 1.01100000 * 2^2$
2. Add significands taking sign of numbers into account: $1.11001110 * 2^2$
3. Normalize to a single leading digit: $1.11001110 * 2^2$
4. Round to 4 fractional bits: $1.1101 * 2^2$

What situations could we run into doing this in hardware with 32-bit floats?

A. Added fraction could be longer than 23 bits
B. Normalized exponent could be greater than 127 or less than -126
C. Shifting fraction to match largest exponent could take more than 23 bits
D. The inputs could be zero or the result could be zero
E. More than one of the above [which?]

Floating point addition algorithm

Input: two single-precision, floating point numbers x, and y

Output: x + y

1. If either x or y is 0, return the other one
2. Denormalize x or y to give them both the larger exponent (use 64-bit integers to hold the significands; hidden bit + 23-bit fraction shifted to the left by 32 bits)
3. Add the significands (as 64-bit integers), taking sign into account
4. If the result is 0, return 0
5. Normalize the result by shifting the added significands left/right and increasing/decreasing the exponent
   Ex: $10011.101 * 2^{-1} = 1001.1101 * 2^{0} = 100.11101 * 2^{1}$

In JavaScript, you perform the operation 9007199254740992 + 1. What is the result?

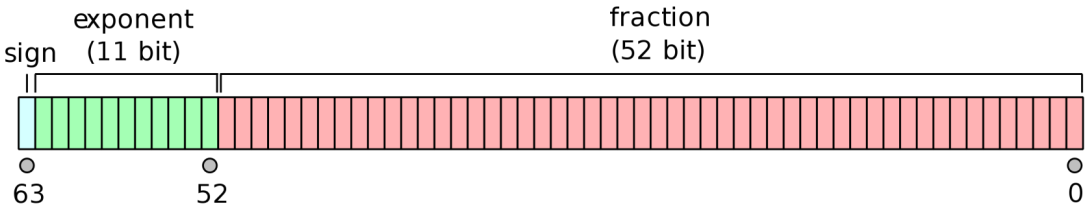
A. -9007199254740992

B. 9007199254740992

C. 9007199254740993

D. This will cause an error

E. None of the above



Reminder: 9007199254740992 is 2^{53}

How many times will this loop run in python?

```
a = 1000
while a != 0:
   a -= 0.001
```

A. 1,000 times
B. 100,000 times
C. 1,000,000 times
D. It will run forever
E. None of the above

This will run forever

```
a = 1000
while a != 0:
   a -= 0.001
```

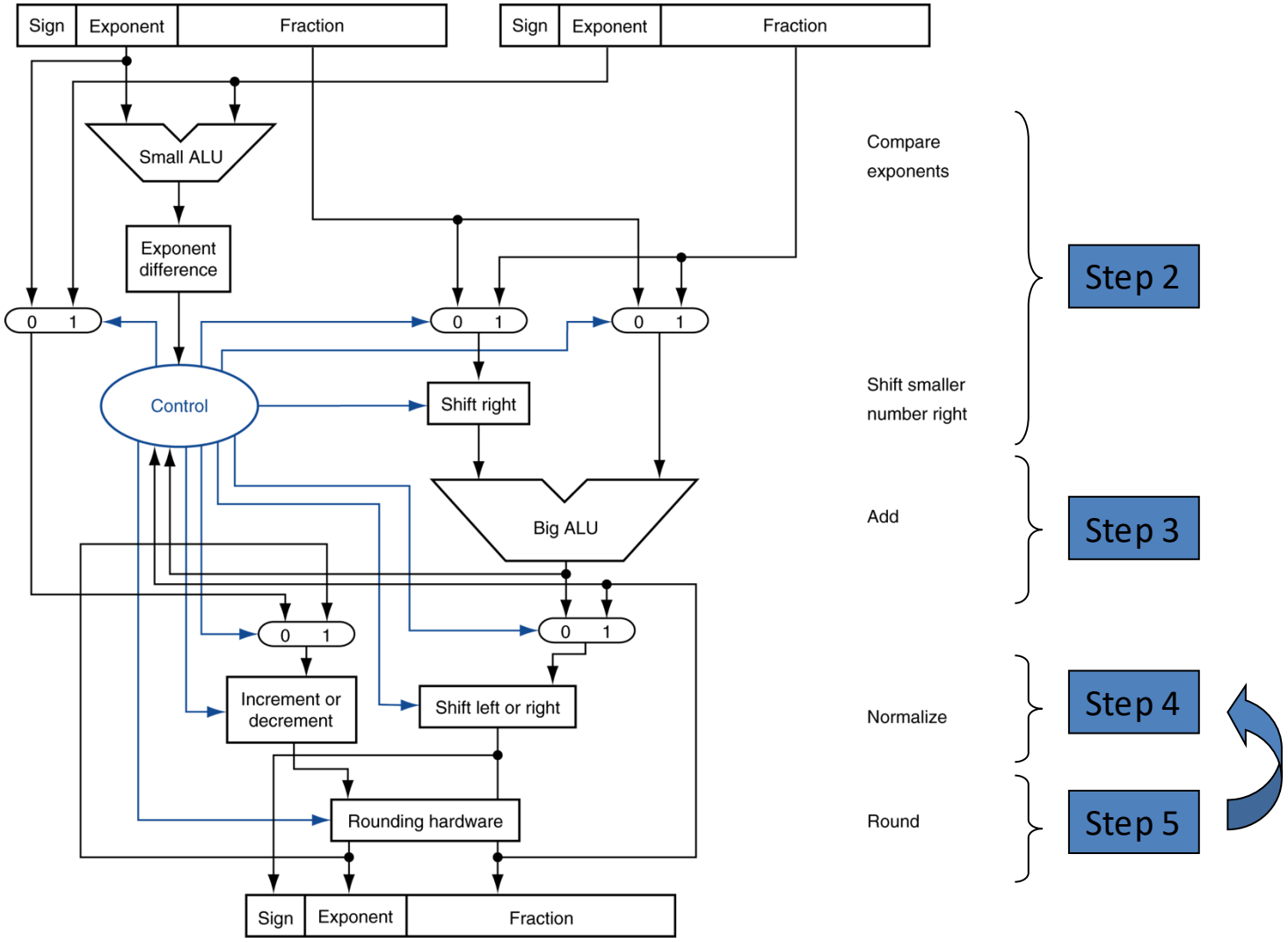
- a is never 0, instead it goes from 1.6734946768626l9e-08 to -0.000999983265053231 4.

- Takeaway: Float equality is hard! Usually want to check within a small range (but this is hard to get right too!)
  - I.e., don't do x == y, instead abs(x – y) < epsilon for some small epsilon, but picking the right epsilon is tricky

FP Adder Hardware

- Much more complex than integer adder
- Doing it in the general purpose ALU would take too long
  - Much longer than integer operations
  - Slower clock would penalize all instructions
- FP adder usually takes several cycles (but there are techniques we're going to learn about later that can deal with this)

FP Adder Hardware



Reading

- Next lecture: Finish floating point; RISC-V addressing